

VGGT: Visual Geometry Grounded Transformer

KAIST LAVA LAB
Hyeshim Kim

Contents

Overview	—————	Positioning	<u>Preliminary</u>	Method	—————	Validation	—————	Discussion
				• Overview • Problem Definition • Feature Backbone • Prediction Heads				

Overview

Paper info

Journal/Conference: **2025 CVPR (Best Paper)**

Keywords: **3D Reconstruction, Feed-Forward, Multi-Task Model**

Authors/Institution: **Wang et al. / Oxford, Meta AI**



The key issue that the above paper focuses on

Geometric Post-processing Free →

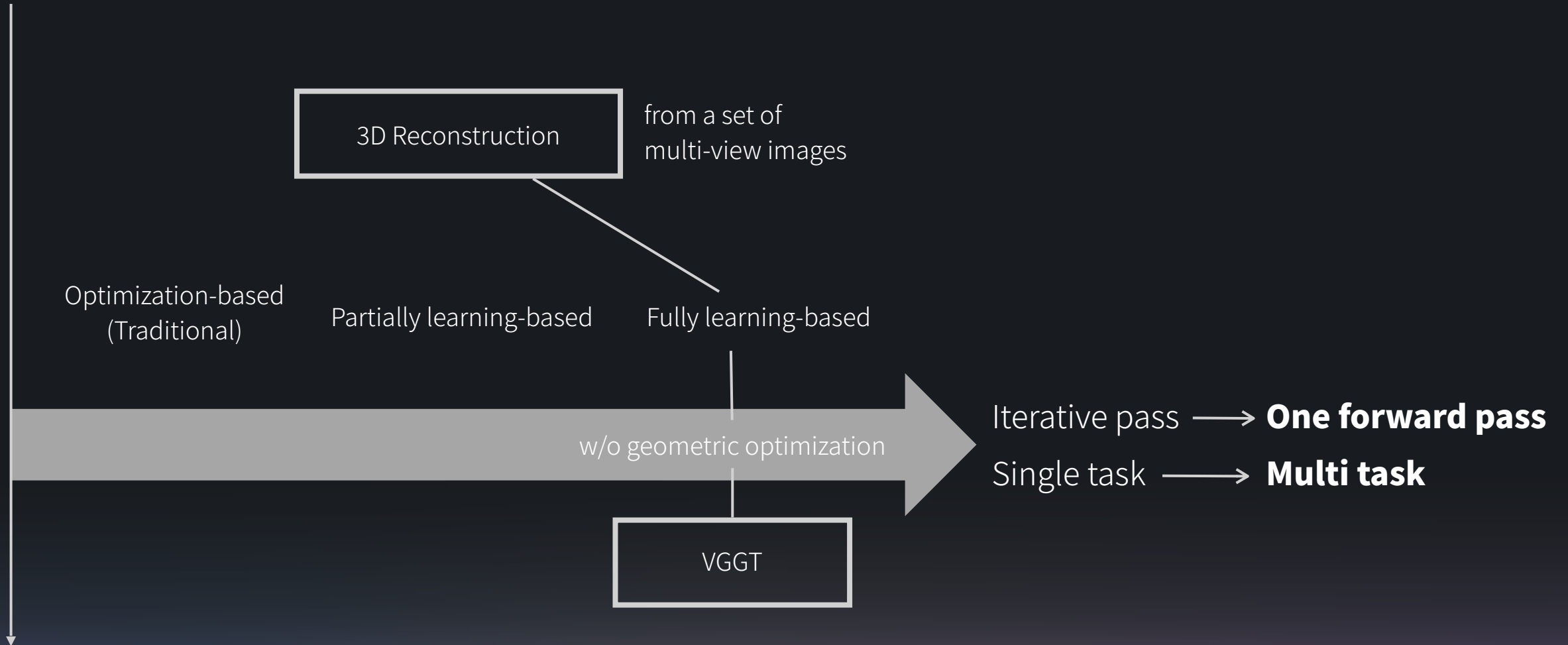
- Feed-forward transformer
- Alternating Attention (frame-wise $\leftrightarrow$ global self-attention)

Unified Full 3D Attribute Prediction →

Camera parameters, depth maps, point maps, 3D point tracks

- Multi-task training with joint losses
- Shared AA transformer trunk, task-specific heads

Positioning

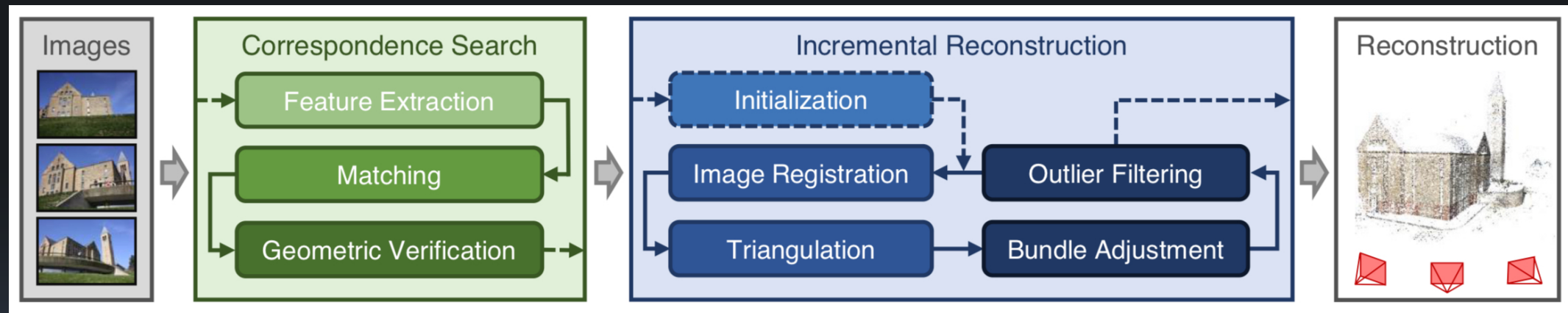


Preliminary

Optimization-based

(Traditional)

e.g., COLMAP



Preliminary

Partially Learning-based

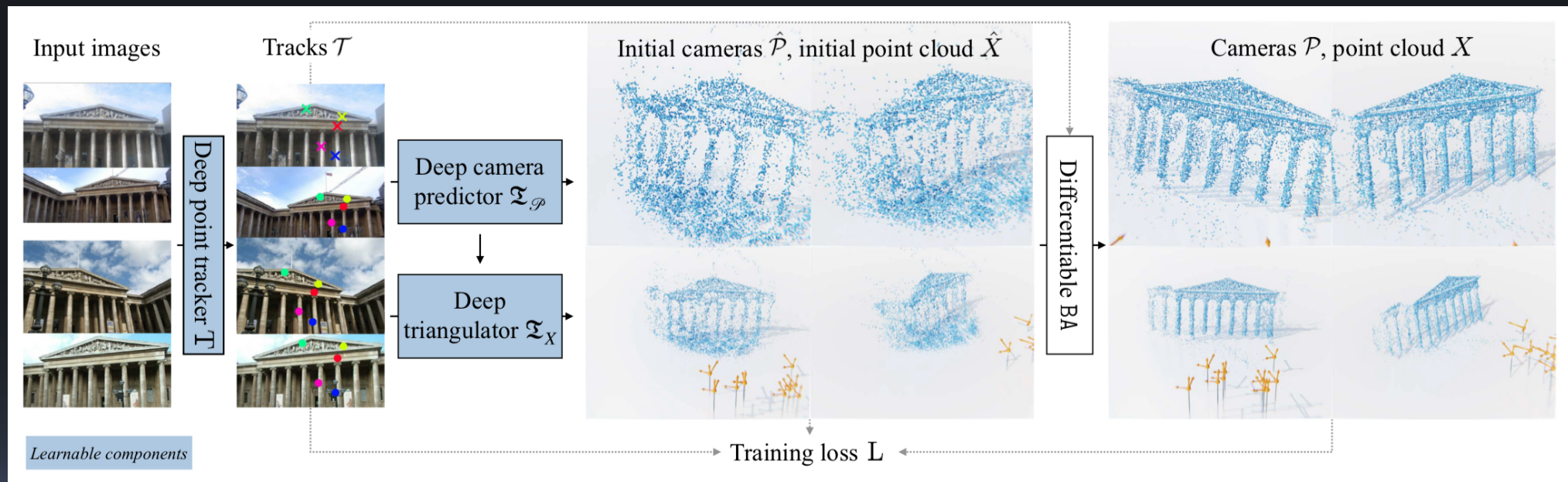
e.g., VGGsfM

Keypoint
detection
(learned)

Feature
matching
(learned)

Bundle
Adjustment
(BA)

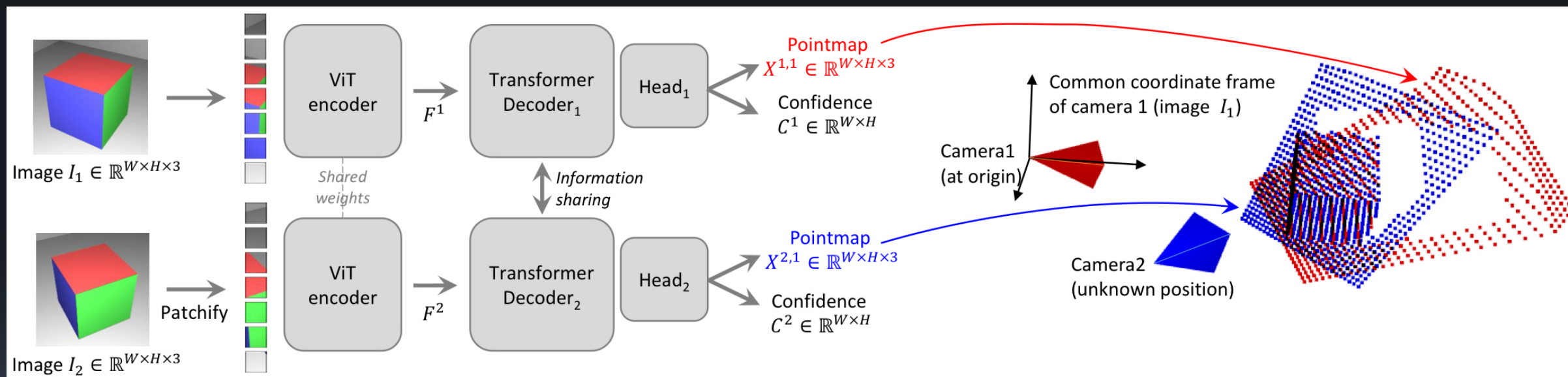
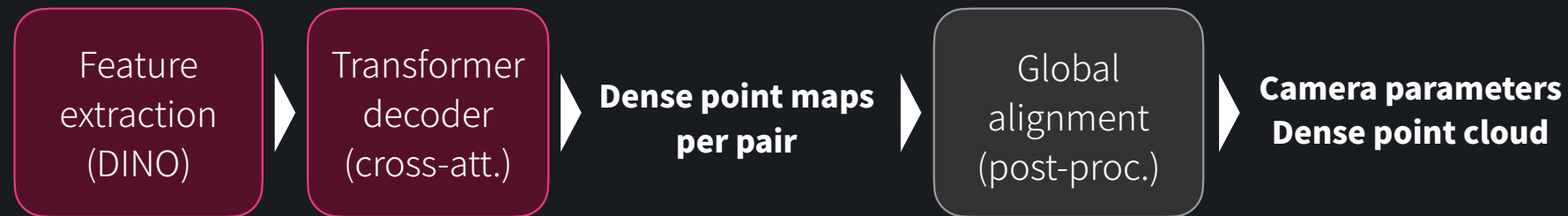
Camera parameters
Sparse point cloud



Preliminary

Fully learning-based

e.g., DUST3R, MAST3R



Method Overview

Fully learning-based

VGGT

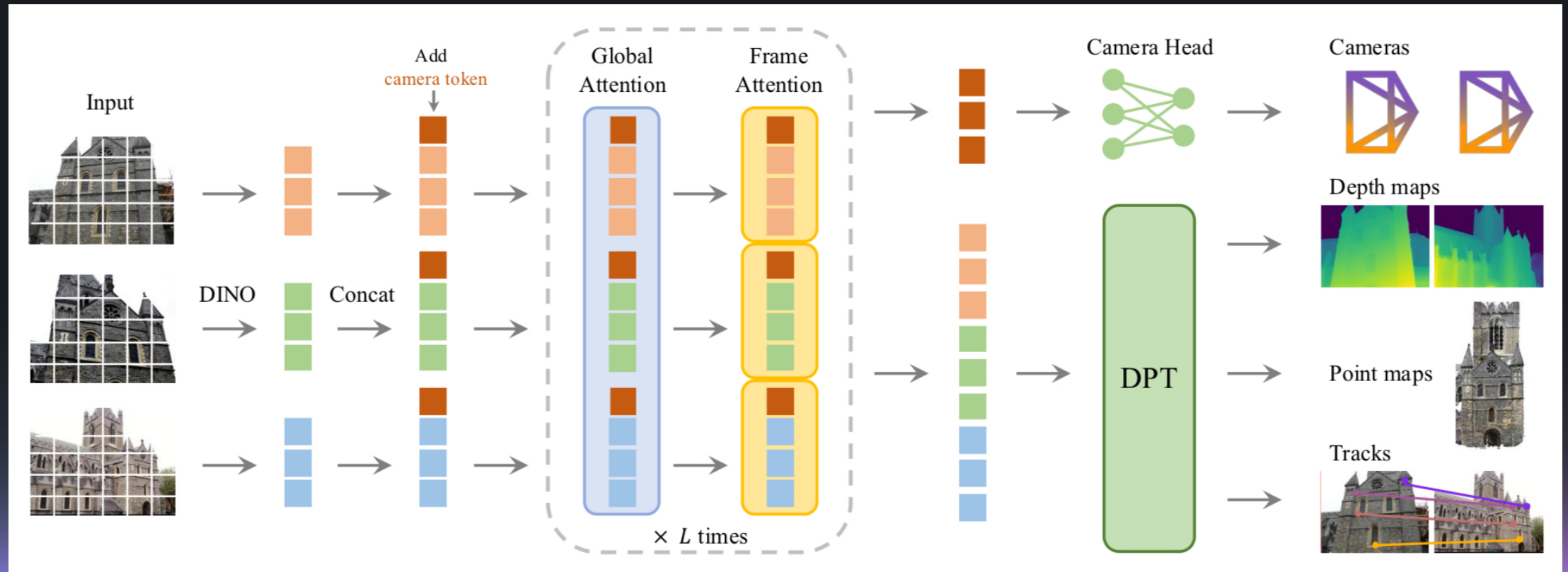
Feed-Forward

Feature
extraction
(DINO)

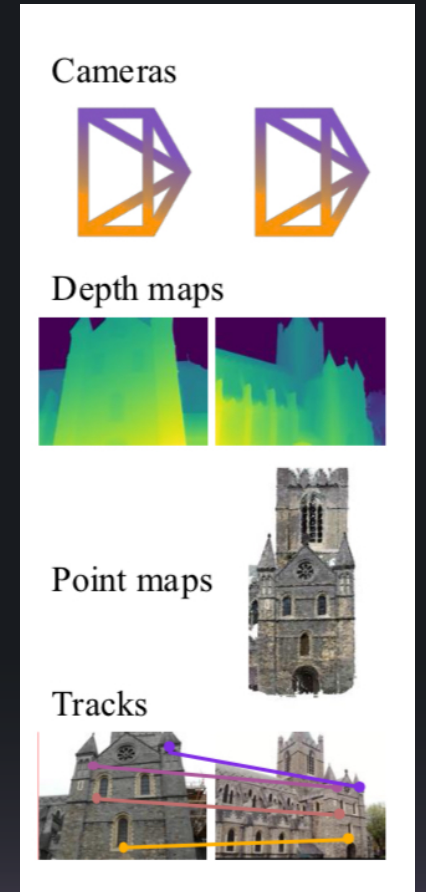
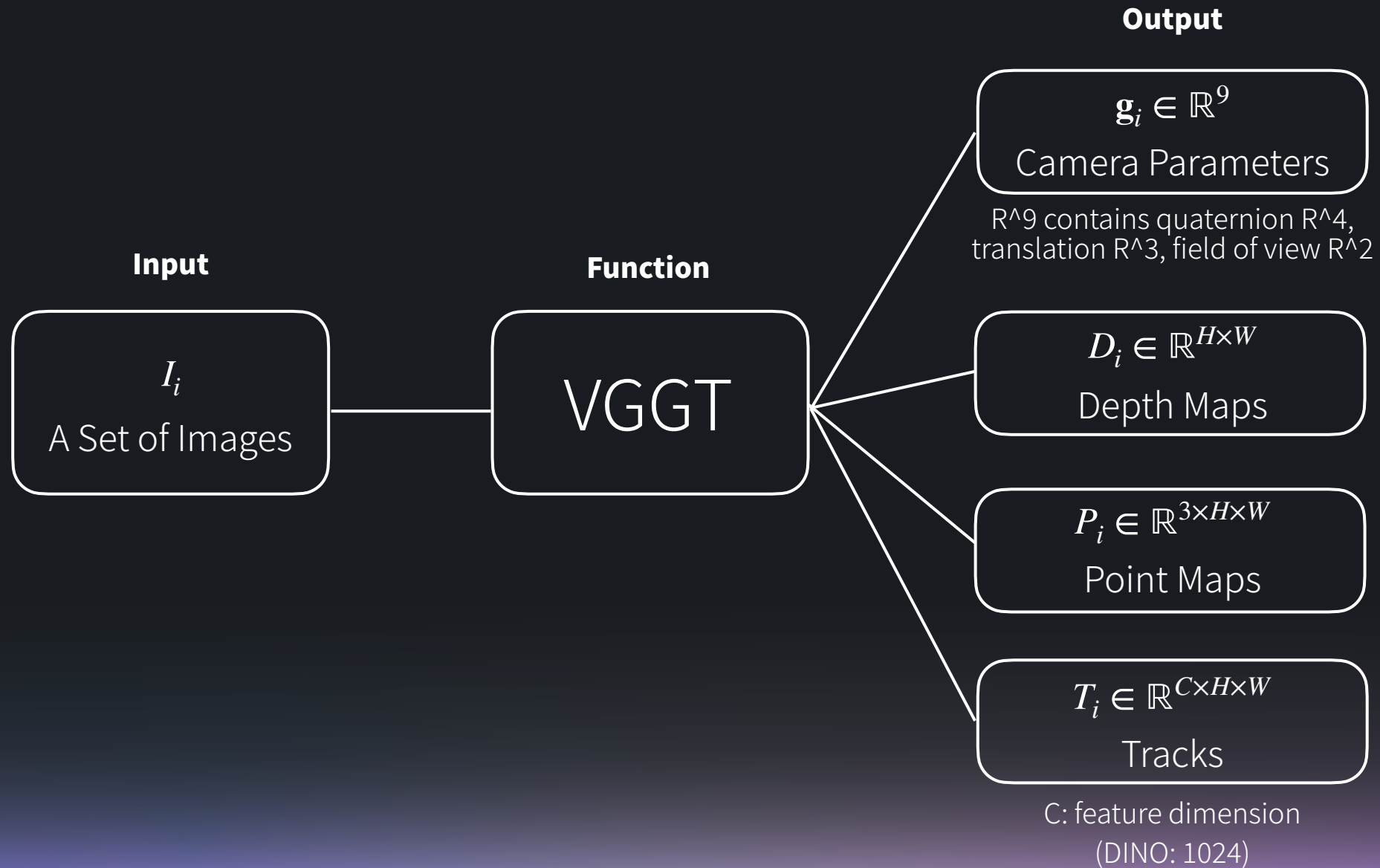
AA
Transformer
(**self-att.**)

Camera parameters
Point maps
Depth maps
Tracks

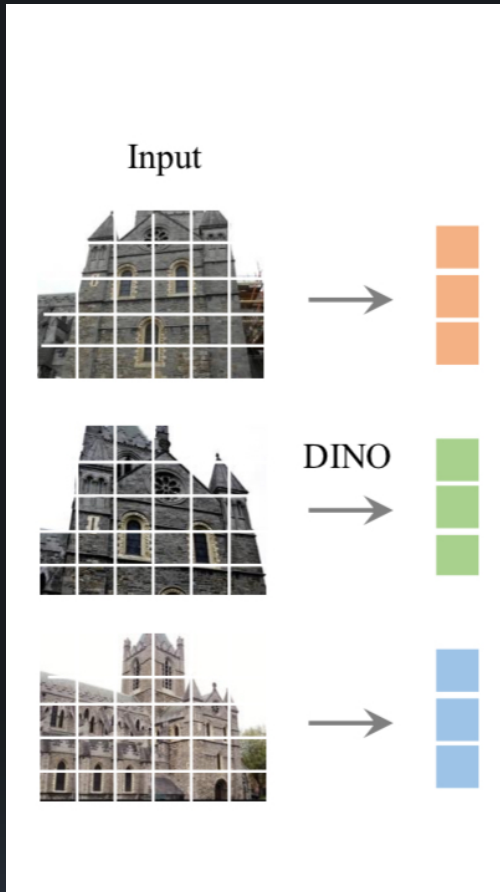
No
Geometric
Post-processing!



Method **Problem Definition**



Method **Feature Backbone - DINO**



14x14
image patch $\rightarrow$ $t_k^I \in \mathbb{R}^C$

**Pre-trained
visual priors**

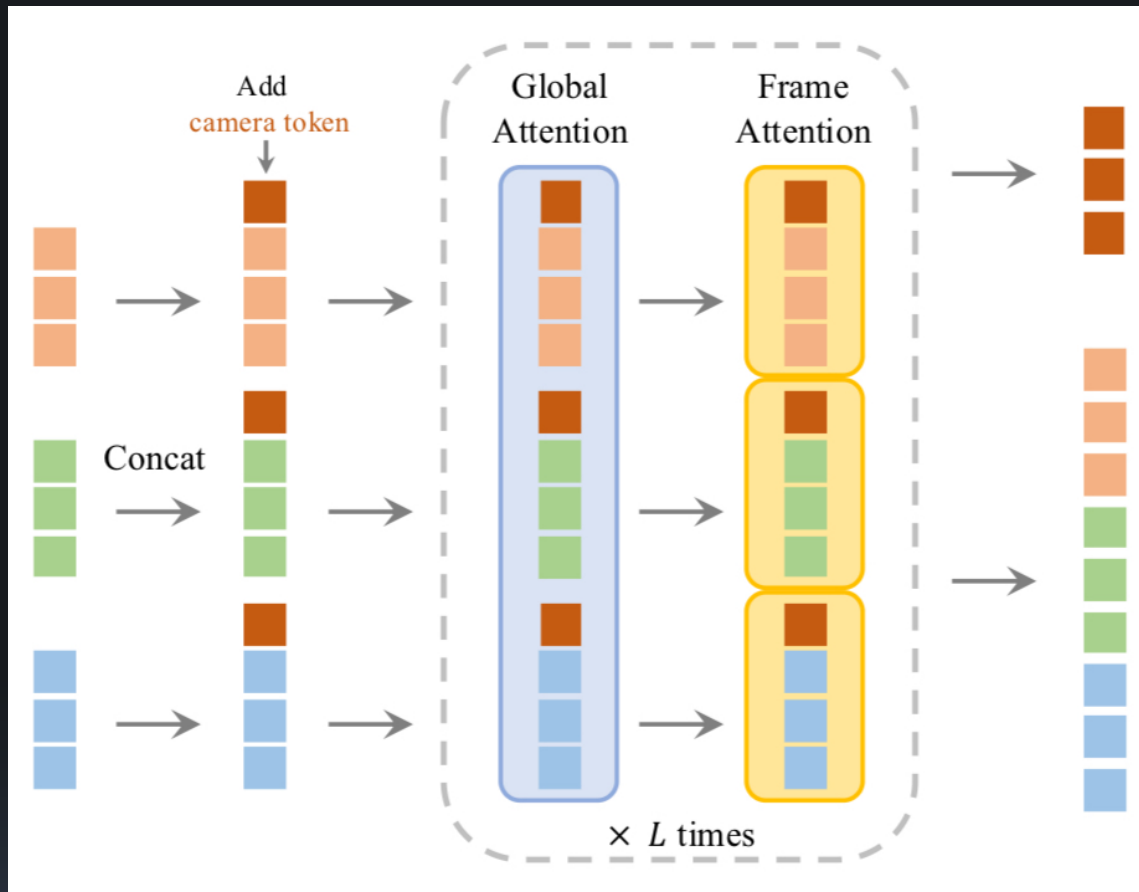
Each feature vector encodes semantic and spatial information of its patch region

t: token, k: num of patch



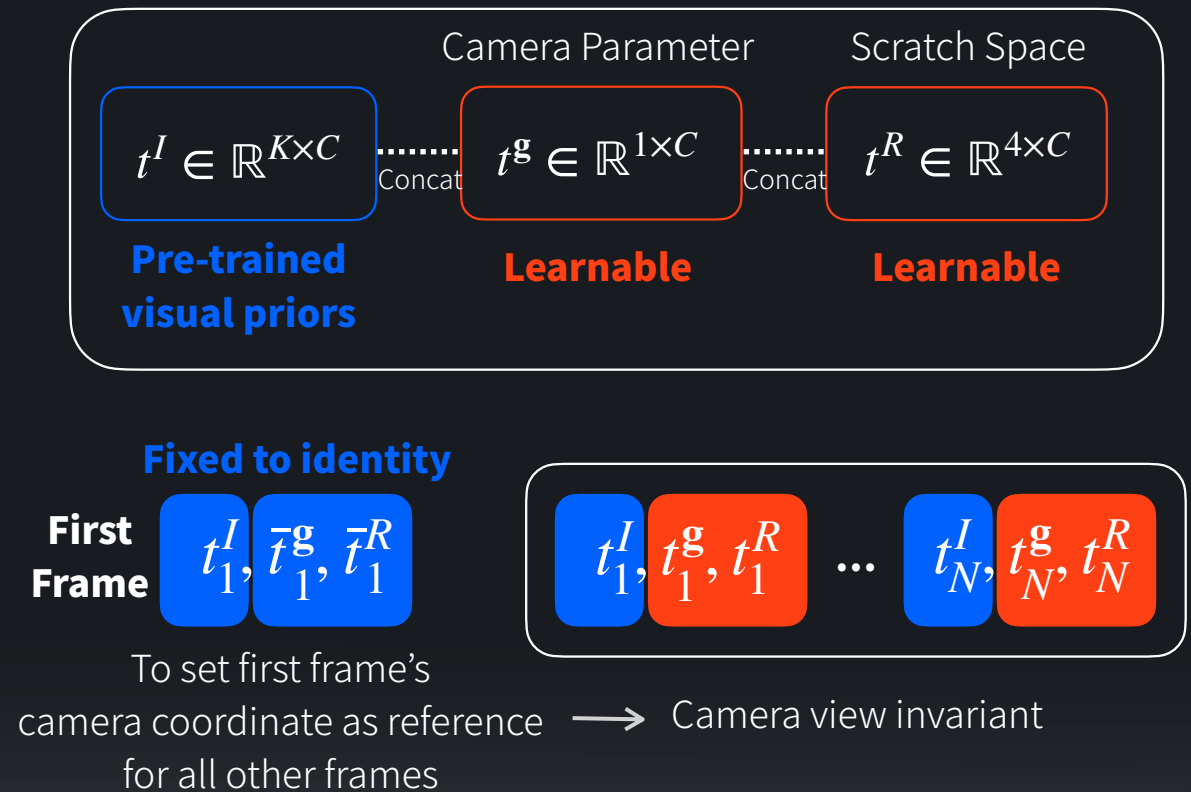
DINO: self-**di**stillation with **no** labels.

Method **Feature Backbone - Alternating-Attention (AA)**



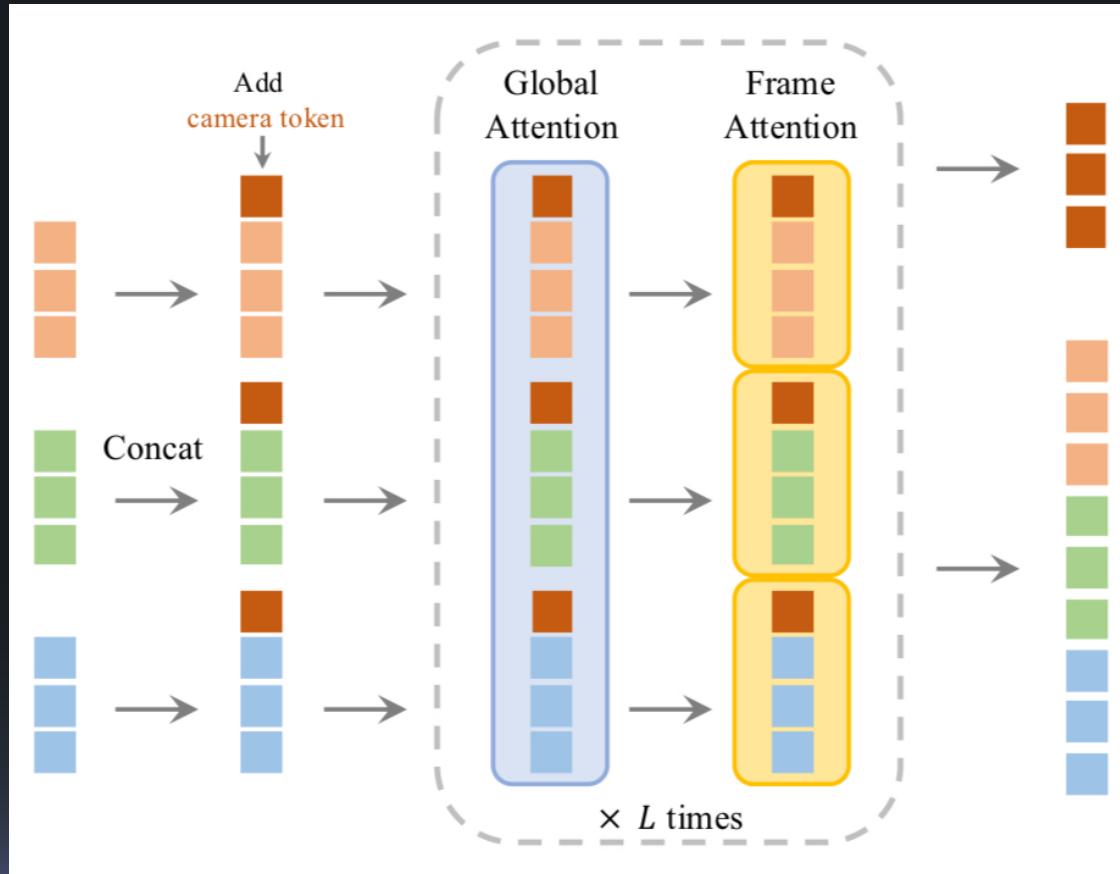
$L=24$

Per Frame



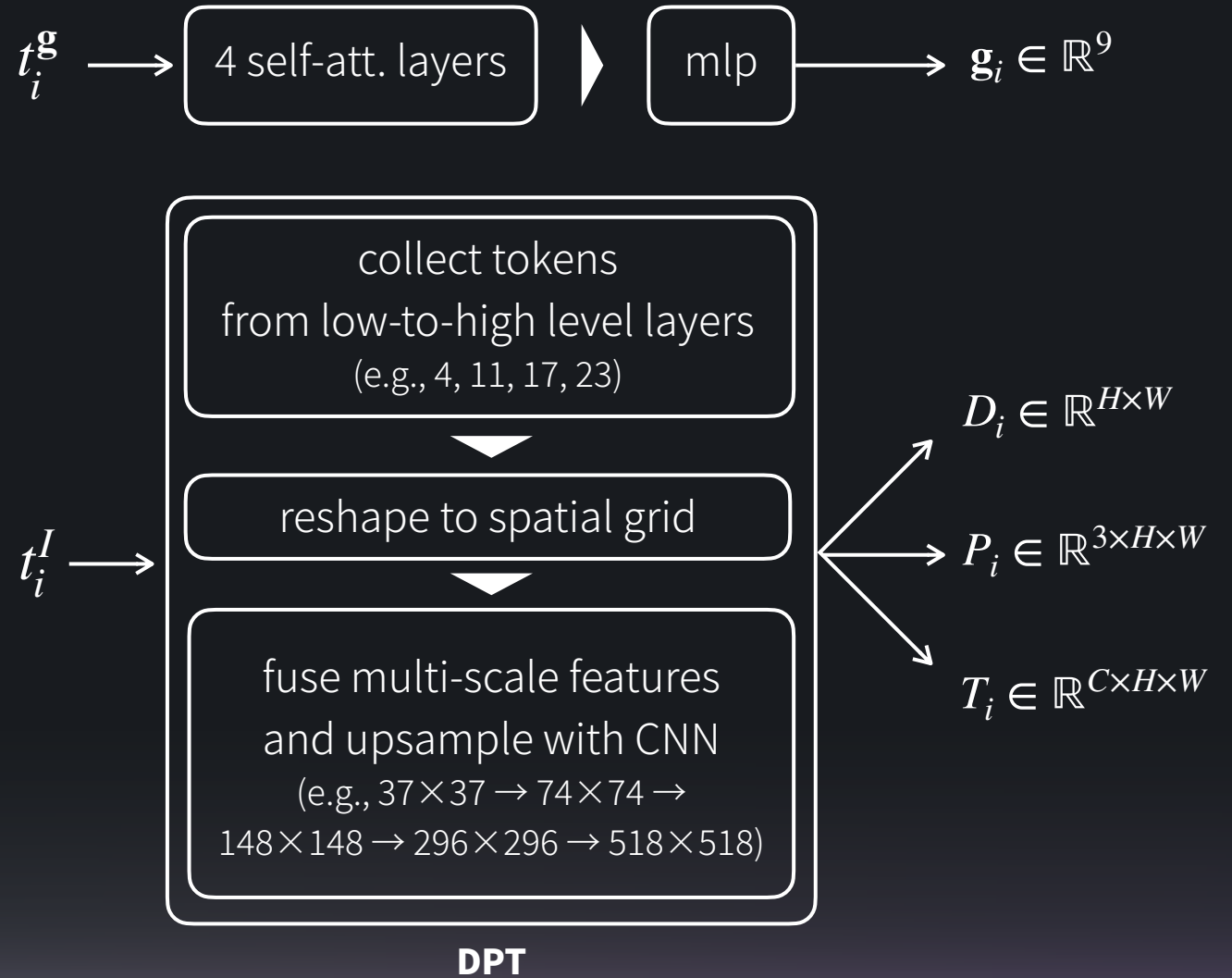
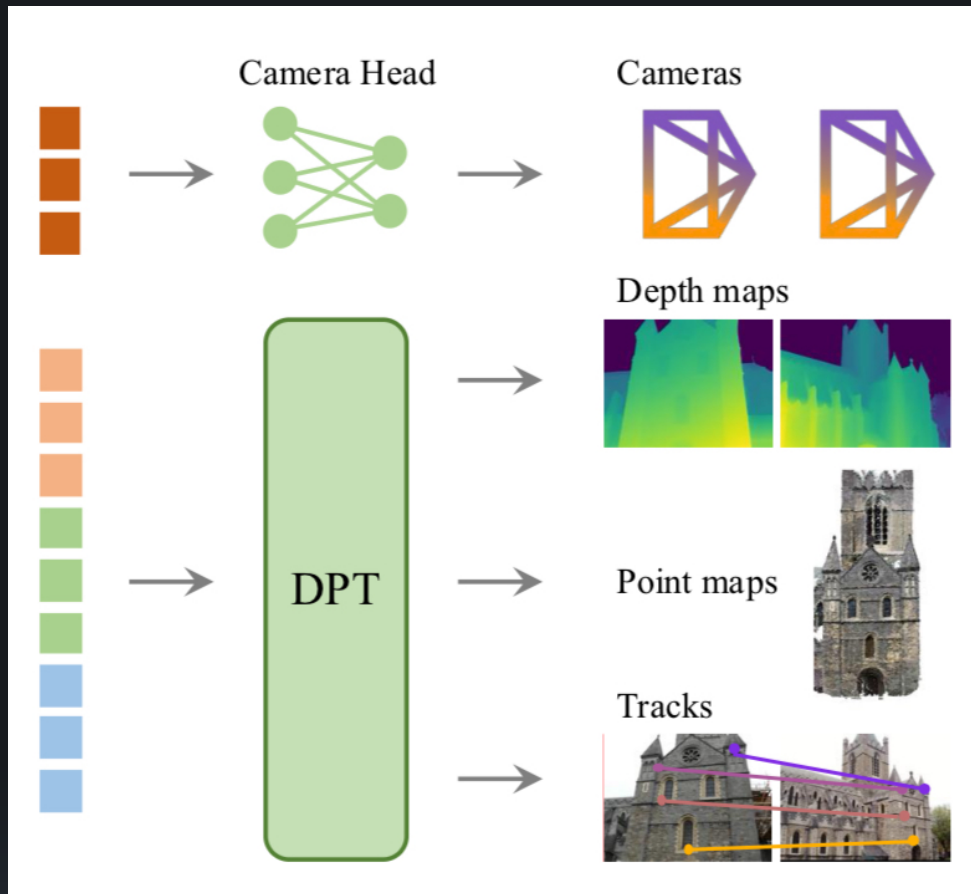
Method **Feature Backbone - Alternating-Attention (AA)**

- Global Attention: cross-view information exchange → **geometric/3D reasoning**
 - Frame Attention: within-view feature refinement → **spatial coherence per image**
- ↩ Balancing

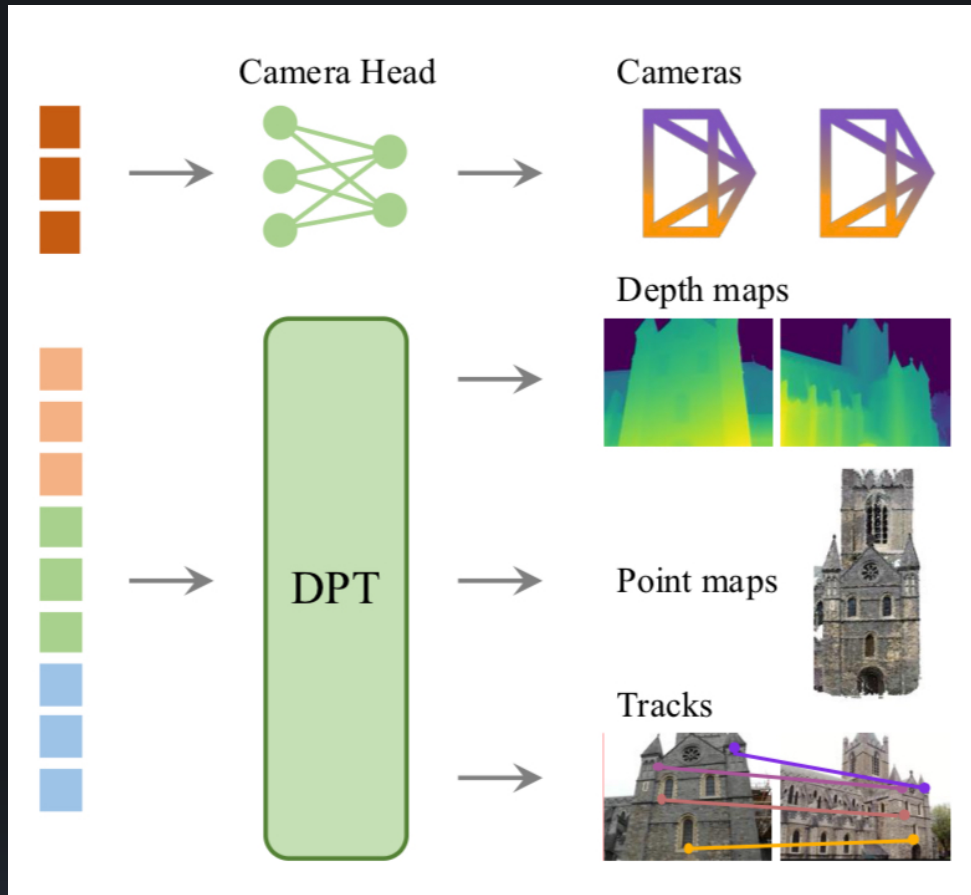


ETH3D Dataset	Acc.↓	Comp.↓	Overall↓
Cross-Attention	1.287	0.835	1.061
Global Self-Attention Only	<u>1.032</u>	<u>0.621</u>	<u>0.827</u>
Alternating-Attention	0.901	0.518	0.709

Method **Prediction Heads**



Method Prediction Heads



Multi-task training with joint losses

$$\mathcal{L} = \mathcal{L}_{\text{camera}} + \mathcal{L}_{\text{depth}} + \mathcal{L}_{\text{pmap}} + \lambda \mathcal{L}_{\text{track}}$$

lambda=0.05

w. $\mathcal{L}_{\text{camera}}$	w. $\mathcal{L}_{\text{depth}}$	w. $\mathcal{L}_{\text{track}}$	Acc.↓	Comp.↓	Overall↓
✗	✓	✓	1.042	0.627	0.834
✓	✗	✓	<u>0.920</u>	<u>0.534</u>	<u>0.727</u>
✓	✓	✗	0.976	0.603	0.790
✓	✓	✓	0.901	0.518	0.709

- $\mathcal{L}_{\text{camera}}$: predicted vs GT camera params (Huber loss)
- $\mathcal{L}_{\text{depth}}$: predicted vs GT depth (with uncertainty weighting)
- $\mathcal{L}_{\text{pmap}}$: predicted vs GT point maps (with uncertainty weighting)
- $\mathcal{L}_{\text{track}}$: predicted vs GT 2D correspondences
- **Training with all losses jointly improves every individual task**

Validation

Camera Pose Estimation

Methods	Re10K (<i>unseen</i>) AUC@30 ↑	CO3Dv2 AUC@30 ↑	Time
Colmap+SPSG [60]	45.2	25.3	~ 15s
PixSfM [45]	49.4	30.1	> 20s
PoseDiff [82]	48.0	66.5	~ 7s
DUST3R [87]	67.7	76.7	~ 7s
MASt3R [43]	76.4	81.8	~ 9s
VGGSfM v2 [83]	78.9	83.4	~ 10s
MV-DUST3R [73] ‡	71.3	(69.5)	~ 0.6s
CUT3R [85] ‡	75.3	82.8	~ 0.6s
FLARE [105] ‡	78.8	83.4	~ 0.5s
Fast3R [96] ‡	72.7	82.5	~ 0.2s
Ours (Feed-Forward)	<u>85.3</u>	<u>88.2</u>	~ 0.2s
Ours (with BA)	93.5	91.8	~ 1.8s

Table 1. **Camera Pose Estimation on RealEstate10K [109] and CO3Dv2 [58]** with 10 random frames. All metrics the higher the better. Runtime were measured using one H100 GPU. None of the methods were trained on the Re10K dataset; () means not trained on CO3D. Methods marked with ‡ represent concurrent work.

- Outperforms post-optimization methods (DUST3R, MASt3R, VGGSfM)
- Generalizes well to unseen datasets (RealEstate10K, not in training)

Multi-view Depth Estimation

Known GT camera	Method	Acc.↓	Comp.↓	Overall↓
✓	Gipuma [28]	0.283	0.873	0.578
✓	MVSNet [98]	0.396	0.527	0.462
✓	CIDER [94]	0.417	0.437	0.427
✓	PatchmatchNet [80]	0.427	0.377	0.417
✓	MASt3R [43]	0.403	0.344	0.374
✓	GeoMVSNet [106]	0.331	0.259	0.295
✗	DUST3R [87]	2.677	0.805	1.741
✗	Ours	0.389	0.374	0.382

Table 2. **Dense MVS Estimation on the DTU [35] Dataset.** Methods operating with known ground-truth camera are in the top part of the table, while the bottom part contains the methods that do not know the ground-truth camera.

- Outperforms DUST3R (Overall: 0.382 vs 1.741)
- Comparable to methods that do know ground-truth cameras

Validation

Point Map Estimation

Methods	Acc.↓	Comp.↓	Overall↓	Time
DUS3R	1.167	0.842	1.005	~ 7s
MASt3R	0.968	0.684	0.826	~ 9s
Ours (Point)	<u>0.901</u>	<u>0.518</u>	<u>0.709</u>	~ 0.2s
Ours (Depth + Cam)	0.873	0.482	0.677	~ 0.2s

Table 3. **Point Map Estimation on ETH3D [65]**. DUS3R and MASt3R use global alignment while ours is feed-forward and, hence, much faster. The row *Ours (Point)* indicates the results using the point map head directly, while *Ours (Depth + Cam)* denotes constructing point clouds from the depth map head combined with the camera head.

- Depth head + Camera head → better point clouds than the dedicated Point Map head (decomposing the task helps)

Multi-view Depth Estimation

Method	AUC@5 ↑	AUC@10 ↑	AUC@20 ↑
SuperGlue [60]	16.2	33.8	51.8
LoFTR [69]	22.1	40.8	57.6
DKM [20]	29.4	50.7	68.3
CasMTR [5]	27.1	47.0	64.4
Roma [21]	31.8	53.4	70.9
Ours	33.9	55.2	73.4

Table 4. **Two-View matching comparison on ScanNet-1500 [9, 60]**. Although our tracking head is not specialized for the two-view setting, it outperforms the state-of-the-art two-view matching method Roma. Measured in AUC (higher is better).

- VGGT’s tracking head beats dedicated two-view matchers like Roma
- Not even designed for this task and just a byproduct of learned tracking features

Validation **Fine-tuning for Downstream Tasks**

Method	Known Input Cam	Size	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
LGM [72]	✓	256	21.44	0.832	0.122
GS-LRM [103]	✓	256	29.59	0.944	0.051
LVSM [36]	✓	256	31.71	0.957	0.027
Ours-NVS*	✗	224	30.41	0.949	0.033

Table 7. **Quantitative comparisons for view synthesis on GSO [17] dataset.** Finetuning VGGT for feed-forward novel view synthesis, it demonstrates competitive performance even without knowing camera extrinsic and intrinsic parameters for the input images. Note that * indicates using a small training set (only 20%).

NVS: competitive with LVSM without knowing input cameras, using only 20% training data

Method	Kinetics			RGB-S			DAVIS		
	AJ	$\delta_{\text{avg}}^{\text{vis}}$	OA	AJ	$\delta_{\text{avg}}^{\text{vis}}$	OA	AJ	$\delta_{\text{avg}}^{\text{vis}}$	OA
TAPTR [44]	49.0	64.4	85.2	60.8	76.2	87.0	<u>63.0</u>	<u>76.1</u>	<u>91.1</u>
LocoTrack [8]	52.9	66.8	85.3	69.7	<u>83.2</u>	<u>89.5</u>	62.9	75.3	87.2
BootsTAPIR [16]	<u>54.6</u>	<u>68.4</u>	<u>86.5</u>	<u>70.8</u>	83.0	89.9	61.4	73.6	88.7
CoTracker [38]	49.6	64.3	83.3	67.4	78.9	85.2	61.8	76.1	88.3
CoTracker + Ours	57.2	69.0	88.9	72.1	84.0	91.6	64.7	77.5	91.4

Table 8. **Dynamic Point Tracking Results on the TAP-Vid benchmarks.** Although our model was not designed for dynamic scenes, simply fine-tuning CoTracker with our pretrained weights significantly enhances performance, demonstrating the robustness and effectiveness of our learned features.

Dynamic tracking: replacing CoTracker’s backbone with VGGT features significantly boosts performance on TAP-Vid, despite VGGT being trained only on static scenes

Discussion

Reasons for Acceptance of This Paper

- **State-of-the-art performance:** fastest inference (0.2s), feed-forward (no post-processing), highest accuracy
- **Unified multi-task prediction:** camera pose, depth, point maps, image matching, downstream tasks
- **Exceptionally thorough evaluation:** 6+ tasks, multiple datasets per task, ablation studies

Limitations of the paper

- **First frame dependency:** the choice of “first” image affects all outputs, breaking permutation-equivariance
- **Memory scaling:** 40GB GPU memory for 200 frames, meaning there’s a practical upper limit
- **DINOv2 dependency:** if DINO features are weak for a certain domain VGGT likely struggles too